S T Q Q S S D

__/__/__

1) $A = \begin{bmatrix} 10 & -1 & 2 \\ -1 & 11 & -1 \\ 2 & -1 & 10 \end{bmatrix}$

$x = \begin{bmatrix} x \\ y \\ z \end{bmatrix}$ $b = \begin{bmatrix} 6 \\ 22 \\ -10 \end{bmatrix}$

$$\begin{bmatrix} 10 & -1 & 2 \\ -1 & 11 & -1 \\ 2 & -1 & 10 \end{bmatrix} \begin{bmatrix} x \\ y \\ z \end{bmatrix} = \begin{bmatrix} 6 \\ 22 \\ -10 \end{bmatrix}$$

2)

linha 1

$$|10| > |-1| + |2|$$
$$10 > 1 + 2 = 3$$

linha 2

$$|11| > |-1| + |-1|$$
$$11 > 1 + 1 = 2$$

linha 3

$$|10| > |2| + |-1|$$
$$10 > 2 + 1 = 3$$

A matriz é estritamente diagonalmente dominante por linhas

__/__/__

S T Q Q S S D

3) $10x - y + 2z = 6$

$$x^{(k)} = \frac{6 + y^{(k-1)} - 2z^{(k-1)}}{10}$$

2ª equação:

$$-x + 11y - z = 22$$

$$y^{(k)} = \frac{22 + x^{(k)} + z^{(k-1)}}{11}$$

3ª equação:

$$2x - y + 10z = -10$$

$$z^{(k)} = \frac{-10 - 2x^{(k)} + y^{(k)}}{10}$$

4) $(x^0, y^0, z^0) = (0, 0, 0)$

$k = 1$

$$x^{(1)} = \frac{6 + 0 - 2(0)}{10} = 0,6$$

$$y^{(1)} = \frac{22 + 0,6 + 0}{11} = 2,054545$$

$$z^{(1)} = \frac{-10 - 2(0,6) + 2,054545}{10} = -0,914545$$

$k = 2$

$$x^{(2)} = \frac{6 + 2{,}054545 - 2(-0{,}914545)}{10} = 0{,}988364$$

$$y^{(2)} = \frac{22 + 0{,}988364 - 0{,}914545}{11} = 2{,}006711$$

$$z^{(2)} = \frac{-10 - 2(0{,}988364) + 2{,}006711}{10} = -0{,}997002$$

$k = 3$

$$x^{(3)} = \frac{6 + 2{,}006711 - 2(-0{,}997002)}{10} = 1{,}000071$$

$$y^{(3)} = \frac{22 + 1{,}000071 - 0{,}997002}{11} = 2{,}000279$$

$$z^{(3)} = \frac{-10 - 2(1{,}000071) + 2{,}000279}{10} = -0{,}999986$$

Questão 5 — Implementação em Python

Link do Colab: https://colab.research.google.com/drive/11fiRHRvBXO-etwnqBCnTEopKfM09oa5-?usp=sharing

Entrada

```python
import numpy as np
import matplotlib.pyplot as plt

A = np.array([[10, -1, 2],
              [-1, 11, -1],
              [2, -1, 10]], dtype=float)
b = np.array([6, 22, -10], dtype=float)

x = np.zeros(3)
tol = 1e-5
hist = [(0, *x, np.nan)]
k = 0
while True:
    x_old = x.copy()
    for i in range(3):
        s = sum(A[i, j] * x[j] for j in range(3) if j != i)
        x[i] = (b[i] - s) / A[i, i]      # usa valores já atualizados
    k += 1
    err = np.max(np.abs(x - x_old))      # norma infinito
    hist.append((k, *x, err))
    if err < tol:
        break

print("Iterações:", k)
print("Solução aproximada:", x)
print("Solução exata     :", np.linalg.solve(A, b))
```

Saída

```
Iterações: 6
Solução aproximada: [ 1.00000003  1.99999998 -1.00000001]
Solução exata     : [ 1.  2. -1.]
```

Questão 6 — Tabela de iterações

Entrada

```
import pandas as pd
df = pd.DataFrame(hist, columns=["k", "x", "y", "z", "erro"])
df
```

Saída

	k	x	y	z	erro
0	0	0.000000	0.000000	0.000000	NaN
1	1	0.600000	2.054545	-0.914545	2.054545e+00
2	2	0.988364	2.006711	-0.997002	3.883636e-01
3	3	1.000071	2.000279	-0.999986	1.170777e-02
4	4	1.000025	2.000004	-1.000005	2.755404e-04
5	5	1.000001	2.000000	-1.000000	2.389216e-05
6	6	1.000000	2.000000	-1.000000	1.262607e-06

Questão 7 — Gráficos

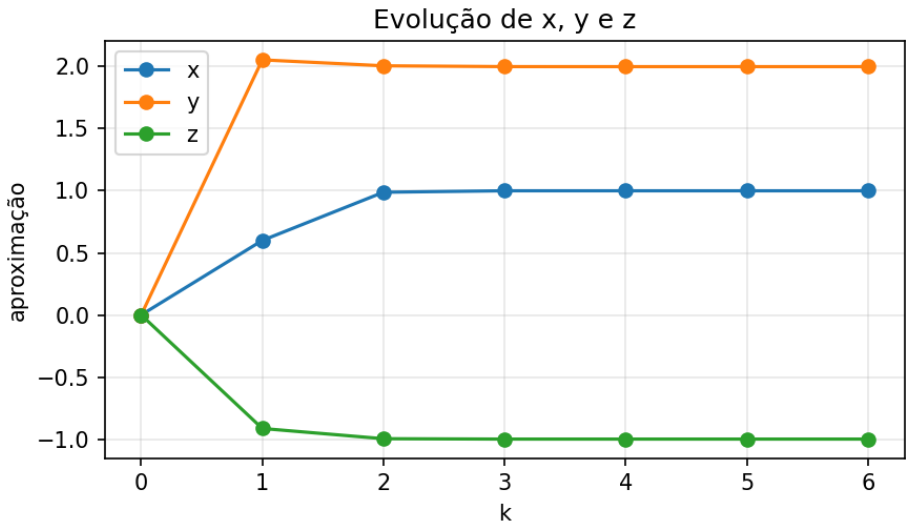
Entrada

```
h = np.array(hist, dtype=float)

# a) x, y, z em função de k
plt.figure()
for c, n in zip((1, 2, 3), "xyz"):
    plt.plot(h[:, 0], h[:, c], "o-", label=n)
plt.xlabel("k"); plt.legend(); plt.grid(alpha=.3); plt.show()

# b) erro entre iterações (escala log)
plt.figure()
plt.semilogy(h[1:, 0], h[1:, 4], "o-")
plt.xlabel("k"); plt.ylabel("||x(k) - x(k-1)||∞"); plt.grid(alpha=.3, which="both"); plt.show()
```

Saída — (a) x, y e z em função de k



Saída — (b) erro entre iterações (escala logarítmica)

